BUBT | BANGLADESH UNIVERSITY OF BUSINESS AND TECHNOLOGY

Computer Science and Engineering

Lab Report

Final

Compiler Design Lab (CSE-324)

Submitted By:
Kaniz Fatema
20245103154
Intake 53 (03)

Submitted To:
Md. Nahid Hossain
Lecturer
Dept. of CSE

August 17, 2026

Contents

1 LL(1) parsing

1.0.1 Question

1. **Write a program to implement LL(1) parsing for the following grammar**

1.0.2 Solution

```cpp
#include <bits/stdc++.h>
using namespace std;

map<char, vector<string>> G;
map<char, set<char>> FIRST, FOLLOW;
map<char, map<char, string>> table;

char startSymbol;

bool isNT(char c)
{
    return c >= 'A' && c <= 'Z';
}

set<char> firstString(string s)
{
    set<char> ans;

    if (s == "#")
    {
        ans.insert('#');
        return ans;
    }

    for (char c : s)
    {
        if (!isNT(c))
        {
            ans.insert(c);
            return ans;
        }

        for (char x : FIRST[c])
            if (x != '#')
                ans.insert(x);

        if (!FIRST[c].count('#'))
            return ans;
    }

    ans.insert('#');
    return ans;
```

```
}

void findFirst()
{
    bool change = true;

    while (change)
    {
        change = false;

        for (auto p : G)
        {
            char A = p.first;

            for (string rhs : p.second)
            {
                set<char> f = firstString(rhs);

                for (char x : f)
                    if (FIRST[A].insert(x).second)
                        change = true;
            }
        }
    }
}

void findFollow()
{
    FOLLOW[startSymbol].insert('$');

    bool change = true;

    while (change)
    {
        change = false;

        for (auto p : G)
        {
            char A = p.first;

            for (string rhs : p.second)
            {
                for (int i = 0; i < rhs.size(); i++)
                {
                    char B = rhs[i];

                    if (!isNT(B))
                        continue;

                    string beta = rhs.substr(i + 1);
```

```
93
94                      if (beta != "")
95                      {
96                          set<char> f = firstString(beta);
97
98                          for (char x : f)
99                          {
100                             if (x != '#')
101                             {
102                                 if (FOLLOW[B].insert(x).second)
103                                     change = true;
104                             }
105                         }
106
107                         if (f.count('#'))
108                         {
109                             for (char x : FOLLOW[A])
110                                 if (FOLLOW[B].insert(x).second)
111                                     change = true;
112                         }
113                     }
114                     else
115                     {
116                         for (char x : FOLLOW[A])
117                             if (FOLLOW[B].insert(x).second)
118                                 change = true;
119                     }
120                 }
121             }
122         }
123     }
124 }
125
126 void makeTable()
127 {
128     for (auto p : G)
129     {
130         char A = p.first;
131
132         for (string rhs : p.second)
133         {
134             set<char> f = firstString(rhs);
135
136             for (char x : f)
137                 if (x != '#')
138                     table[A][x] = rhs;
139
140             if (f.count('#'))
141                 for (char x : FOLLOW[A])
142                     table[A][x] = rhs;
```

```
143         }
144     }
145 }
146 
147 string showStack(stack<char> s)
148 {
149     string x = "";
150 
151     while (!s.empty())
152     {
153         x += s.top();
154         s.pop();
155     }
156 
157     return x;
158 }
159 
160 void parse(string input)
161 {
162     input += '$';
163 
164     stack<char> st;
165     st.push('$');
166     st.push(startSymbol);
167 
168     int i = 0;
169 
170     cout << "\nStack\tInput\tAction\n";
171     cout << "--------------------------------\n";
172 
173     while (!st.empty())
174     {
175         char top = st.top();
176         char current = input[i];
177 
178         cout << showStack(st) << "\t"
179              << input.substr(i) << "\t";
180 
181         if (top == current)
182         {
183             if (top == '$')
184             {
185                 cout << "Accepted\n";
186                 cout << "\nString is accepted\n";
187                 return;
188             }
189 
190             cout << "Match " << current << endl;
191 
192             st.pop();
```

```
            i++;
        }

        else if (isNT(top))
        {
            if (!table[top].count(current))
            {
                cout << "No Rule\n";
                cout << "\nString is not accepted\n";
                return;
            }

            string rhs = table[top][current];

            cout << top << "->" << rhs << endl;

            st.pop();

            if (rhs != "#")
            {
                for (int j = rhs.size() - 1; j >= 0; j--)
                    st.push(rhs[j]);
            }
        }

        else
        {
            cout << "Mismatch\n";
            cout << "\nString is not accepted\n";
            return;
        }
    }
}

int main()
{
    int n;

    cout << "Enter number of productions: ";
    cin >> n;

    for (int i = 0; i < n; i++)
    {
        string p;
        cin >> p;

        char lhs = p[0];
        string rhs = p.substr(3);

        G[lhs].push_back(rhs);
```

```
243
244         if (i == 0)
245             startSymbol = lhs;
246     }
247
248     findFirst();
249     findFollow();
250     makeTable();
251
252     string input;
253
254     cout << "Enter string: ";
255     cin >> input;
256
257     parse(input);
258
259     return 0;
260 }
```

Listing 1: C++ program to LL(1) parsing.

Output

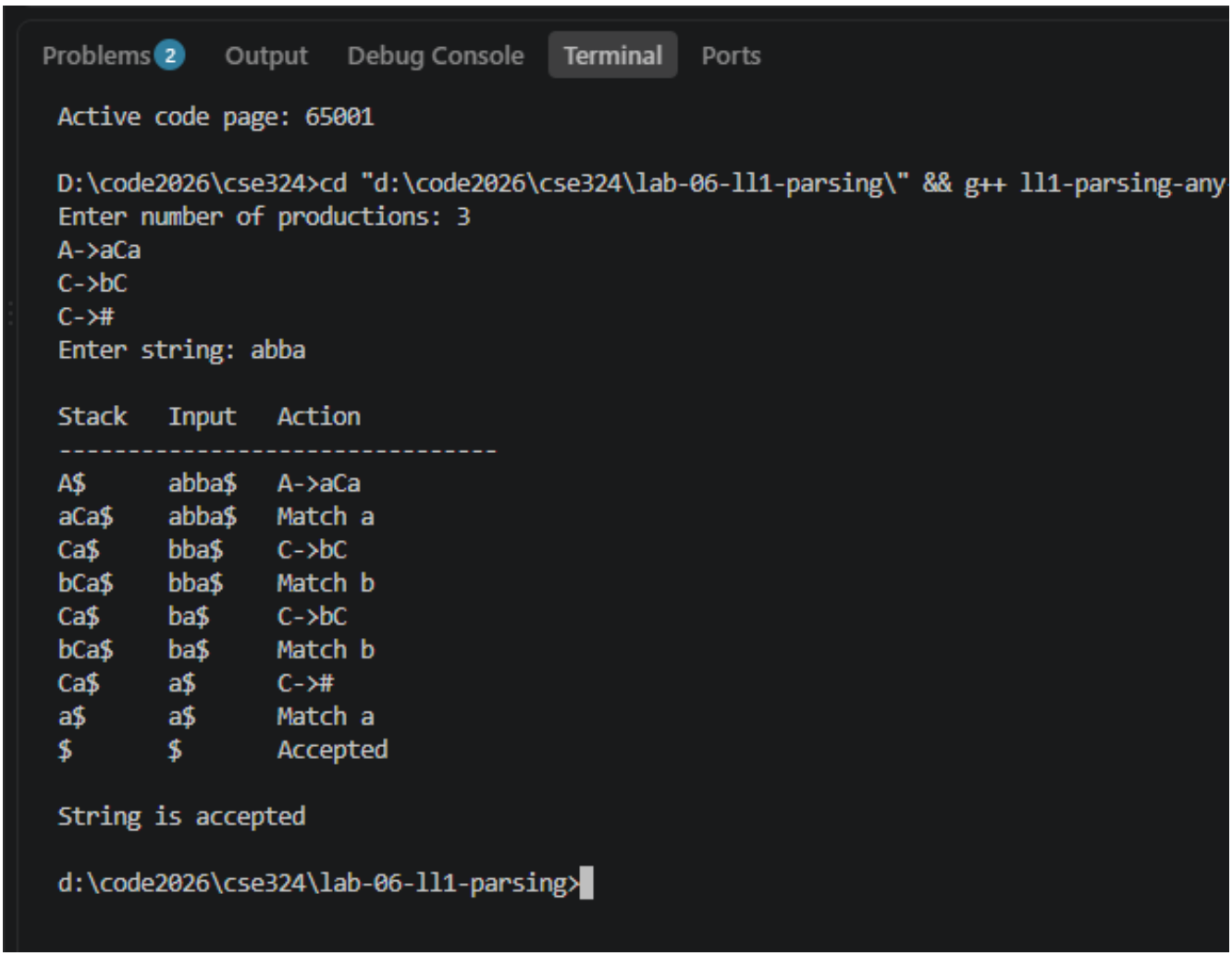

Figure 1: Output of LL(1) parsing

2 Recursive Descent Parsing

2.0.1 Question

1. **Write a program to implement Recursive Descent Parsing for the following grammar.**

Write a program to implement Recursive Descent Parsing for the following grammar

E->TE'

E'->+TE' | €

T-> FT'

T'-> *FT'|€

F -> (E) | a

Sample input: a+a*a

Sample output: String is accepted

Sample input: a/a*a

Sample output: String is not accepted

2.0.2 Solution

```
#include <iostream>
#include <map>
#include <vector>
#include <string>

using namespace std;

map<char, vector<string>> grammar;
string input;

// match a production
bool match(string rule, int pos, int rulePos, int &endPos);

```

```
14 // Try a non-terminal
15 bool parse(char nt, int pos, int &endPos)
16 {
17     for (string rule : grammar[nt])
18     {
19         int tempPos;
20 
21         // Epsilon
22         if (rule == "#")
23         {
24             endPos = pos;
25             return true;
26         }
27 
28         if (match(rule, pos, 0, tempPos))
29         {
30             endPos = tempPos;
31             return true;
32         }
33     }
34 
35     return false;
36 }
37 
38 // Match symbols of a production
39 bool match(string rule, int pos, int rulePos, int &endPos)
40 {
41     // Production finished
42     if (rulePos == rule.length())
43     {
44         endPos = pos;
45         return true;
46     }
47 
48     char symbol = rule[rulePos];
49 
50     // Non-terminal
51     if (symbol >= 'A' && symbol <= 'Z')
52     {
53         int newPos;
54 
55         if (parse(symbol, pos, newPos))
56         {
57             return match(rule, newPos, rulePos + 1, endPos);
58         }
59 
60         return false;
61     }
62 
63     // Terminal
```

```
64      if (pos < input.length() && input[pos] == symbol)
65      {
66          return match(rule, pos + 1, rulePos + 1, endPos);
67      }
68
69      return false;
70  }
71
72  int main()
73  {
74      int n;
75
76      cout << "Enter number of productions: ";
77      cin >> n;
78
79      cout << "Enter productions separately.\n";
80      cout << "Use # for epsilon.\n\n";
81
82      char startSymbol;
83
84      for (int i = 0; i < n; i++)
85      {
86          string p;
87
88          cout << "Production " << i + 1 << ": ";
89          cin >> p;
90
91          char lhs = p[0];
92          string rhs = p.substr(3);
93
94          grammar[lhs].push_back(rhs);
95
96          if (i == 0)
97              startSymbol = lhs;
98      }
99
100     cout << "\nEnter input string: ";
101     cin >> input;
102
103     int endPos = 0;
104
105     if (parse(startSymbol, 0, endPos) &&
106         endPos == input.length())
107     {
108         cout << "String is accepted";
109     }
110     else
111     {
112         cout << "String is not accepted";
113     }
```

```
114
115     return 0;
116 }
```

Listing 2: LEX/C++ program to Recursive Descent Parsing.

Output

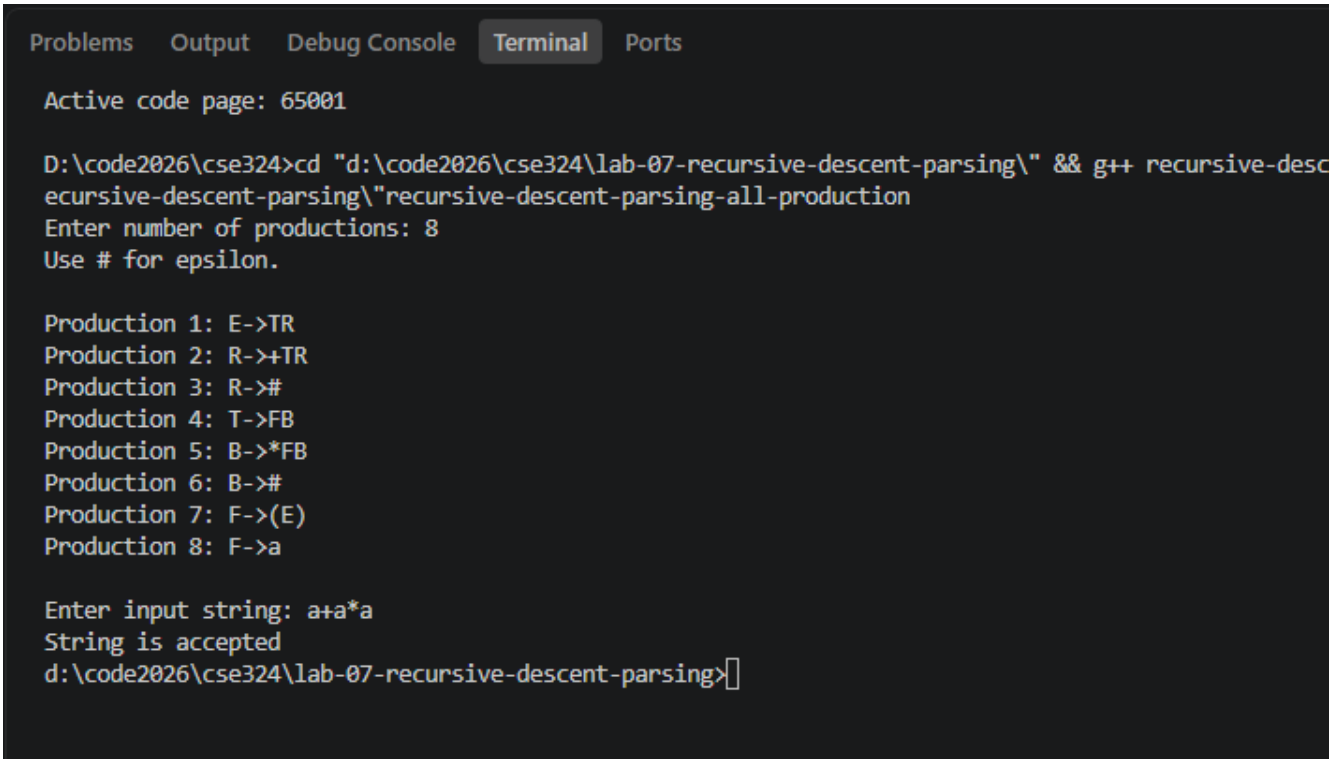

Figure 2: Output of Recursive Descent Parsing

3 Predictive parsing table

3.0.1 Question

1. **Write a program to implement Predictive parsing table for the following grammar.**

Write a C/ C++ program to implement Predictive parsing table for mini language.

Sample Input:

E -> eEx

E -> aBc

F -> fFy

G -> gGz

Sample Output:

	e	a	f	g
E	E->eEx	E->aBc		
F			F->fFy	
G				gGz

3.0.2 Solution

```cpp
#include <iostream>
#include <string>
#include <vector>
#include <map>
#include <set>
#include <cctype>
#include <iomanip>

using namespace std;

map<char, vector<string>> grammar;
map<char, set<char>> firstSet;
map<char, set<char>> followSet;
set<char> nonTerminals;
set<char> terminals;
char startSymbol;

void computeFirst(char c) {
    if (!firstSet[c].empty()) return;

    for (const string& prod : grammar[c]) {
        if (prod == "#") {
            firstSet[c].insert('#');
            continue;
        }

        for (size_t i = 0; i < prod.length(); ++i) {
            char nextChar = prod[i];

            if (!isupper(nextChar)) {
                firstSet[c].insert(nextChar);
                break;
            } else {
                computeFirst(nextChar);
                bool hasEpsilon = false;
                for (char f : firstSet[nextChar]) {
                    if (f == '#') hasEpsilon = true;
                    else firstSet[c].insert(f);
                }
                if (!hasEpsilon) break;
                if (i == prod.length() - 1) firstSet[c].insert('#');
            }
        }
    }
}

void computeFollow() {
    followSet[startSymbol].insert('$');

    bool changed = true;
```

```
    while (changed) {
        changed = false;
        for (char nt : nonTerminals) {
            for (const string& prod : grammar[nt]) {
                for (size_t i = 0; i < prod.length(); ++i) {
                    char c = prod[i];
                    if (isupper(c)) {
                        size_t oldSize = followSet[c].size();

                        if (i + 1 < prod.length()) {
                            char nextChar = prod[i + 1];
                            if (!isupper(nextChar)) {
                                followSet[c].insert(nextChar);
                            } else {
                                bool hasEpsilon = false;
                                for (char f : firstSet[nextChar]) {
                                    if (f == '#') hasEpsilon = true;
                                    else followSet[c].insert(f);
                                }
                                if (hasEpsilon) {
                                    for (char f : followSet[nt])
                                    ↪  followSet[c].insert(f);
                                }
                            }
                        } else {
                            for (char f : followSet[nt]) followSet[c].insert(f);
                        }

                        if (followSet[c].size() != oldSize) changed = true;
                    }
                }
            }
        }
    }
}

int main() {
    int numProds;
    cout << "Enter number of productions: ";
    cin >> numProds;

    cout << "Enter productions:\n";
    for (int i = 0; i < numProds; ++i) {
        string prodLine;
        cin >> prodLine;

        char lhs = prodLine[0];
        string rhs = prodLine.substr(3);

        if (i == 0) startSymbol = lhs;
```

```
100
101         grammar[lhs].push_back(rhs);
102         nonTerminals.insert(lhs);
103
104         for (char ch : rhs) {
105             if (!isupper(ch) && ch != '#') terminals.insert(ch);
106         }
107     }
108     terminals.insert('$');
109
110     for (char nt : nonTerminals) computeFirst(nt);
111     computeFollow();
112
113     // First and Follow Output
114     cout << "\nFIRST AND FOLLOW SETS\n";
115     for (char nt : nonTerminals) {
116         cout << "FIRST(" << nt << ")  = { ";
117         for (char c : firstSet[nt]) cout << c << " ";
118         cout << "}\n";
119
120         cout << "FOLLOW(" << nt << ") = { ";
121         for (char c : followSet[nt]) cout << c << " ";
122         cout << "}\n\n";
123     }
124
125     map<char, map<char, string>> parsingTable;
126     bool isLL1 = true;
127
128     for (char nt : nonTerminals) {
129         for (const string& prod : grammar[nt]) {
130             set<char> firstRHS;
131             if (prod == "#") {
132                 firstRHS.insert('#');
133             } else {
134                 for (size_t i = 0; i < prod.length(); ++i) {
135                     char nextChar = prod[i];
136                     if (!isupper(nextChar)) {
137                         firstRHS.insert(nextChar);
138                         break;
139                     } else {
140                         bool hasEpsilon = false;
141                         for (char f : firstSet[nextChar]) {
142                             if (f == '#') hasEpsilon = true;
143                             else firstRHS.insert(f);
144                         }
145                         if (!hasEpsilon) break;
146                         if (i == prod.length() - 1) firstRHS.insert('#');
147                     }
148                 }
149             }
```

```
150
151             for (char t : firstRHS) {
152                 if (t != '#') {
153                     if (!parsingTable[nt][t].empty()) isLL1 = false;
154                     parsingTable[nt][t] = prod;
155                 } else {
156                     for (char f : followSet[nt]) {
157                         if (!parsingTable[nt][f].empty()) isLL1 = false;
158                         parsingTable[nt][f] =  "#";
159                     }
160                 }
161             }
162         }
163     }
164
165     // LL(1) Parsing Table Output
166     cout << "LL(1) PARSING TABLE\n";
167     cout << left << setw(8) << "NT";
168     for (char t : terminals) {
169         cout << setw(12) << t;
170     }
171     cout << "\n" << string(8 + terminals.size() * 12, '-') << "\n";
172
173     for (char nt : nonTerminals) {
174         cout << setw(8) << nt;
175         for (char t : terminals) {
176             string entry = "-";
177             if (!parsingTable[nt][t].empty()) {
178                 entry = string(1, nt) + "->" + parsingTable[nt][t];
179             }
180             cout << setw(12) << entry;
181         }
182         cout << "\n";
183     }
184
185     if (!isLL1) {
186         cout << "\nGrammar is NOT LL(1) due to structural table conflicts!\n";
187     } else {
188         cout << "\nGrammar is successfully LL(1)!\n";
189     }
190     return 0;
191 }
```

Listing 3: LEX/C++ program to Predictive parsing table.

Output

Figure 3: Output of Predictive parsing table

4 Remove single Line comment

4.0.1 Question

1. **Write a C/C++ program to remove single Line comment from the input file (input.txt) source program..**

4.0.2 Solution

```
/*Write a C/C Program to remove single line comment from the source program.
It reads charecter from the input.txt and write the result to output.txt file.*/

#include<iostream>
#include<fstream>
#include<string>
using namespace std;

int main()
{
    //=========Write File===========
    ofstream myFile("input.txt");

    if(myFile.is_open())
    {
        myFile << "My name is Kaniz Fatema.\n";
        myFile << "//Kaniz Fatema.\n";
        myFile << "/*Kaniz Fatema.*/\n";
        myFile << "/*20245103154.*/\n";
        myFile << "Bangladesh is my country. //I live in Bangladesh.\n";

        myFile.close();
    }

    //=========Read File===========

    ifstream inFile("input.txt");

    if(!inFile.is_open())
    {
        cout << "Error: input.txt file could not be opened!" << endl;
        return 1;
    }

    //=========Output File===========
    ofstream outFile("output.txt");

    string line;

    while(getline(inFile, line))
    {
```

```
        size_t pos = line.find("//");

        if (pos != string::npos)
        {

            string cleanLine = line.substr(0, pos);
            outFile << cleanLine << endl;
        }
        else
        {
            outFile << line << endl;
        }
    }


    inFile.close();
    outFile.close();

    cout << "Single-line comments removed successfully!" << endl;

    return 0;
}
```

Listing 4: LEX/C++ program to remove single Line comment.

Input File

Figure 4: Output of remove single Line comment

Output File

README.md remove-single-line-comments.cpp output.txt U

lab-09-remove-single line-comments > output.txt

```
1  My name is Kaniz Fatema.
2
3  /*Kaniz Fatema.*/
4  /*20245103154.*/
5  Bangladesh is my country.
6
```

Figure 5: Output of remove single Line comment

5 Three address code

5.0.1 Question

1. **Write a C/C++ program to implement three address code**

5.0.2 Solution

```cpp
#include <iostream>
#include <string>
#include <vector>

using namespace std;

int main() {
    string expr = "a+b-c*d";
    int tempCount = 1;

    cout << "Input Expression: " << expr << endl;
    cout << "\nThree Address Code:" << endl;

    string arg1 = string(1, expr[0]);

    for (size_t i = 1; i < expr.length(); i += 2) {
        char op = expr[i];
        string arg2 = string(1, expr[i + 1]);
```

```
        string res = "t" + to_string(tempCount++);

        cout << res << " = " << arg1 << " " << op << " " << arg2 << endl;

        arg1 = res;
    }

    return 0;
}
```

Listing 5: C/C++ program toThree address code.

Output

Figure 6: Output of Three address code program

1 2

[1]Kaniz Fatema
[2]ID: 20245103154